UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura · E.P. Ingeniería de Sistemas

Informe técnico

Simulador Interactivo de Campos Eléctricos y Magnéticos en 3D

Curso	Física 2 — Producto final
Integrantes	Esther Landa
	Diego Armas
Docente	Ramiro Manrique Rua
Ciclo	IV
Fecha	Noviembre de 2026

Contenido

Resumen 4
1. Introducción y objetivos 5
1.1 Objetivo general 5
1.2 Objetivos específicos 5
1.3 Cumplimiento de lo solicitado 5
2. Fundamentos físicos implementados 6
2.1 Unidad 1: Electricidad, campo eléctrico y potencial 6
2.2 Unidad 2: Capacitores y dieléctricos 6
2.3 Unidad 3: Corriente eléctrica y ley de Ohm 6
2.4 Unidad 4: Fuerza y campo magnético 7
3. Arquitectura del software 8
3.1 Visión general 8
3.2 Estructura del repositorio 8
3.3 Decisiones de diseño 9
3.4 Flujo de una interacción 9
3.5 Interfaz y experiencia de usuario 9
4. Algoritmos utilizados 10
4.1 Líneas de campo eléctrico 10
4.2 Equipotenciales: marching squares y marching tetrahedra 10
4.3 Integración de trayectorias: Dormand–Prince 5(4) 10
4.4 Integrador de Boris 10
4.5 Circuitos 10
4.6 Campo magnético: Biot–Savart numérico y líneas de B 11
4.7 Mapas de potencial y color 11
4.8 Complejidad y rendimiento 11
5. Pruebas y validación 12
5.1 Estrategia 12
5.2 Resultados numéricos de validación 12
5.3 Evidencia gráfica 13
5.4 Validación cruzada JavaScript ⇄ Python 15
5.5 Verificación de la interfaz 16
6. Resultados: el simulador en funcionamiento 17
7. Limitaciones y trabajo futuro 19

8. Conclusiones 20
Competencias desarrolladas 20
Referencias 21

Resumen

Este informe describe el diseño, la implementación y la validación del **Simulador Interactivo de Campos Eléctricos y Magnéticos en 3D**, producto final del curso de Física 2. El simulador es una aplicación web (JavaScript, Three.js y WebGL) organizada en las cuatro unidades del curso —campo y potencial eléctrico, capacitores y dieléctricos, corriente eléctrica y ley de Ohm, y fuerza y campo magnético— mediante nueve módulos interactivos. El motor físico es propio, está separado de la interfaz y se verifica con 19 pruebas automáticas en JavaScript y 25 en Python.

Para garantizar la corrección de los resultados se desarrolló, en paralelo, un paquete Python (**fisica_em**, NumPy/SciPy) con implementaciones independientes de cada modelo. Las pruebas contrastan ambos motores entre sí y con soluciones analíticas: la parábola de una carga en campo uniforme coincide con la teoría con error relativo del orden de 10^{-15}; el radio de Larmor y la deriva E×B simulados coinciden con la teoría con diferencias menores que 10^{-7} y 10^{-8} respectivamente; el integrador de Boris conserva la energía en campo magnético y converge con orden 2.00; el balance de potencia de los circuitos es exacto a precisión de máquina.

Palabras clave: campo eléctrico, potencial, ley de Coulomb, Biot–Savart, fuerza de Lorentz, simulación numérica, Three.js, validación.

1. Introducción y objetivos

Los conceptos de campo eléctrico y magnético son abstractos y difíciles de visualizar con lápiz y papel. Un simulador interactivo permite explorar cualitativa y cuantitativamente cómo varían las magnitudes físicas al modificar los parámetros del problema, y a la vez constituye un proyecto de ingeniería de software que integra modelamiento matemático, programación científica, visualización y diseño de interfaces.

1.1 Objetivo general

Desarrollar un simulador interactivo, en 2D y 3D, que visualice en tiempo real campos eléctricos y magnéticos, potenciales, trayectorias de partículas cargadas y circuitos eléctricos, con resultados numéricos verificables.

1.2 Objetivos específicos

1. Implementar los nueve módulos solicitados para las cuatro unidades del curso.
2. Construir un motor físico modular, independiente de la interfaz y con pruebas automáticas.
3. Validar numéricamente los modelos frente a soluciones analíticas y a una implementación independiente en Python.
4. Diseñar una interfaz clara para usuarios no técnicos, con lecturas numéricas, gráficas y animaciones.
5. Documentar el producto: código fuente, manual de usuario e informe técnico.

1.3 Cumplimiento de lo solicitado

Tabla 1. *Correspondencia entre lo solicitado y lo entregado.*

Elemento del documento del producto final	Módulo / entregable	Tecnología
U1 · Visualizador de cargas puntuales (hasta 10 cargas, 2D/3D, líneas y equipotenciales)	«Visualizador de cargas»	JavaScript + Three.js
U1 · Calculadora de potencial	«Calculadora de potencial»	JavaScript; verificación en Python (NumPy)
U1 · Simulador de trayectoria	«Simulador de trayectoria»	JavaScript (RK45); verificación con SciPy
U2 · Simulador de capacitores	«Simulador de capacitores»	JavaScript + Three.js
U2 · Calculadora de energía almacenada	«Energía almacenada»	JavaScript; verificación en Python
U3 · Simulador de circuitos (serie, paralelo, mixto)	«Simulador de circuitos»	JavaScript; verificación con análisis nodal (Python)
U3 · Visualizador de ley de Ohm en tiempo real	«Ley de Ohm»	JavaScript + Canvas 2D
U4 · Simulador de campo magnético	«Campo magnético»	JavaScript + Three.js; Biot–Savart
U4 · Simulador de fuerza de Lorentz (E y B cruzados)	«Fuerza de Lorentz»	JavaScript (Boris/RK45); verificación con SciPy
Código fuente (repositorio)	Carpeta del proyecto con README	Git/GitHub (estructura lista para publicar)
Manual de usuario (PDF)	Manual_de_usuario.pdf	—
Informe técnico	Este documento	—
Video demostrativo y defensa en clase	A cargo del equipo	—

2. Fundamentos físicos implementados

Todas las magnitudes se calculan en unidades del Sistema Internacional. A continuación se resumen los modelos de cada unidad.

2.1 Unidad 1: Electricidad, campo eléctrico y potencial

Ley de Coulomb y superposición. El campo eléctrico de un conjunto de cargas puntuales q_i ubicadas en r_i es la suma vectorial de los campos individuales:

$$\vec{E}(\vec{r}) = k\sum_i q_i \frac{\vec{r}-\vec{r}_i}{|\vec{r}-\vec{r}_i|^3}, \qquad k = \frac{1}{4\pi\varepsilon_0} \tag{1}$$

El potencial eléctrico (cero en el infinito) y la energía potencial del sistema son:

$$V(\vec{r}) = k\sum_i \frac{q_i}{|\vec{r}-\vec{r}_i|}, \qquad U = \sum_{i<j} k\frac{q_i q_j}{r_{ij}}, \qquad \vec{E} = -\nabla V \tag{2}$$

Las **superficies equipotenciales** son los conjuntos V = cte y son perpendiculares a las líneas de campo. Las **líneas de campo** son curvas tangentes a E en cada punto: dr/ds = E/|E|.

Movimiento de una partícula cargada. En un campo eléctrico la segunda ley de Newton da:

$$m\frac{d^2\vec{r}}{dt^2} = q\,\vec{E}(\vec{r}), \qquad \Delta K = -q\,\Delta V \tag{3}$$

Para E uniforme la solución es $r(t) = r_0 + v_0 t + \tfrac{1}{2}(qE/m)t^2$, una parábola. El teorema trabajo–energía ($\Delta K = -q\Delta V$) se usa como verificación numérica en cualquier campo.

2.2 Unidad 2: Capacitores y dieléctricos

$$C = \kappa\,\varepsilon_0\frac{A}{d}, \qquad U = \tfrac{1}{2}CV^2 = \frac{Q^2}{2C} = \tfrac{1}{2}QV, \qquad u_E = \tfrac{1}{2}\kappa\varepsilon_0 E^2 \tag{4}$$

Con un dieléctrico insertado una fracción x de la longitud de las placas, el capacitor equivale a dos capacitores en paralelo, uno con dieléctrico y otro con aire:

$$C(x) = C_0[1 + (\kappa - 1)\,x] \tag{5}$$

Con la **batería conectada** V es constante (Q = CV); con el capacitor **aislado** Q es constante (V = Q/C). Para redes: en serie $1/C_{eq} = \Sigma\, 1/C_i$ con carga común; en paralelo $C_{eq} = \Sigma C_i$ con voltaje común. La **ruptura dieléctrica** ocurre cuando E = V/d supera la rigidez dieléctrica del medio (aire ≈ 3 MV/m). La carga de un capacitor a través de una resistencia R sigue:

$$v_C(t) = V(1 - e^{-t/RC}), \qquad i(t) = \frac{V}{R}e^{-t/RC}, \qquad \int_0^\infty i^2 R\,dt = \tfrac{1}{2}CV^2 \tag{6}$$

2.3 Unidad 3: Corriente eléctrica y ley de Ohm

$$V = IR, \qquad P = VI = I^2R = \frac{V^2}{R} \tag{7}$$

Para resistores en serie $R_{eq} = \Sigma R_i$ (corriente común, V se divide); en paralelo $1/R_{eq} = \Sigma 1/R_i$ (voltaje común, I se divide). Los circuitos mixtos se reducen de adentro hacia afuera y se reparten de afuera hacia adentro. Las leyes de Kirchhoff (nodos y mallas) y la conservación de la energía ($P_{fuente} = \Sigma I^2 R$) se usan como verificaciones.

2.4 Unidad 4: Fuerza y campo magnético

Ley de Biot–Savart. El campo magnético de una corriente estacionaria es:

$$\vec{B}(\vec{r}) = \frac{\mu_0}{4\pi} \oint \frac{I\, d\vec{l} \times \hat{r}}{r^2} \tag{8}$$

Casos analíticos usados como referencia:

$$B_{\text{hilo}} = \frac{\mu_0 I}{2\pi\rho}, \qquad B_{\text{espira, eje}} = \frac{\mu_0 I R^2}{2(R^2+z^2)^{3/2}}, \qquad B_{\text{solenoide}} \approx \mu_0 \frac{N}{L} I, \qquad \frac{F}{\ell} = \frac{\mu_0 I_1 I_2}{2\pi d} \tag{9}$$

Las líneas de B son siempre cerradas ($\nabla \cdot B = 0$). Un **imán de barra** se modela como un solenoide equivalente (corriente superficial de magnetización).

Fuerza de Lorentz. Una partícula de carga q en campos E y B cumple:

$$m \frac{d\vec{v}}{dt} = q\left(\vec{E} + \vec{v} \times \vec{B}\right) \tag{10}$$

En campos uniformes la trayectoria es una hélice de eje paralelo a B superpuesta a la deriva E×B:

$$\omega_c = \frac{|q|B}{m}, \qquad r_L = \frac{m v_\perp}{|q|B}, \qquad \vec{v}_d = \frac{\vec{E} \times \vec{B}}{B^2}, \qquad v = \frac{E}{B} \ \text{(selector)} \tag{11}$$

3. Arquitectura del software

3.1 Visión general

La aplicación sigue una arquitectura **por capas** con dependencias en un solo sentido: la capa de presentación depende de los algoritmos geométricos y del motor físico, nunca al revés. El motor físico (src/physics) está formado por funciones puras sin referencia al DOM ni a Three.js, lo que permite probarlo desde Node.js y compararlo con la implementación en Python.

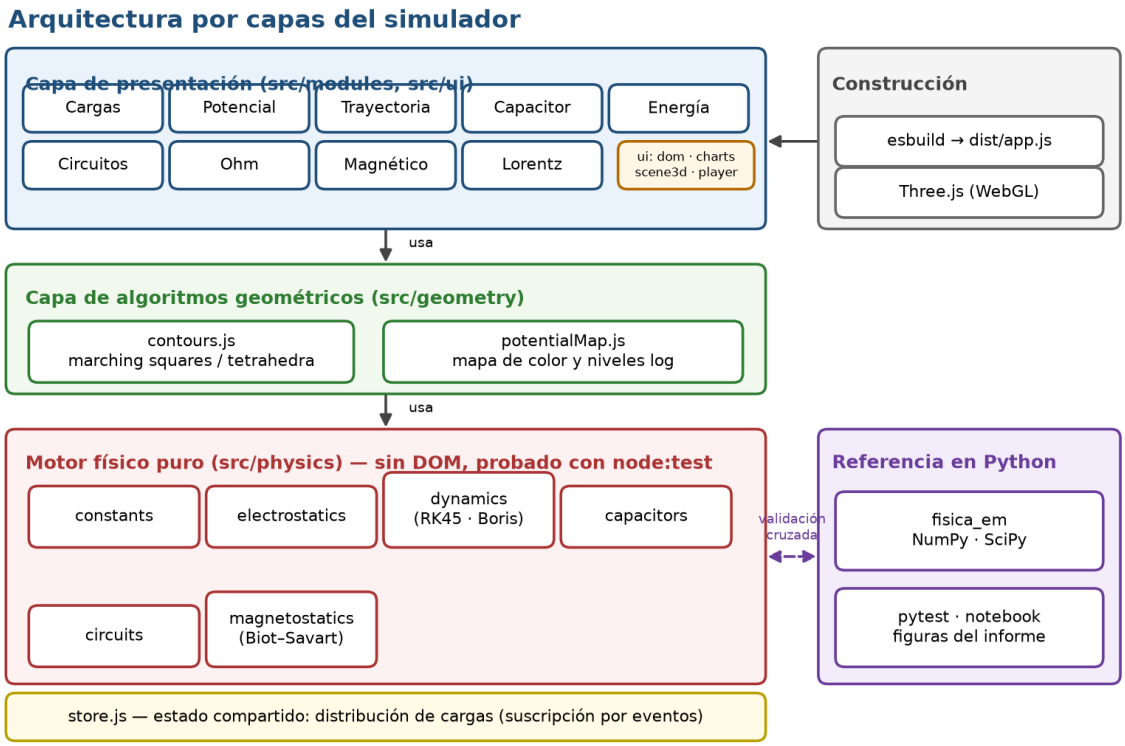


Figura 1. *Arquitectura por capas del simulador y relación con la referencia en Python.*

3.2 Estructura del repositorio

```
Proyecto_Esther/
├── simulador-web/
│   ├── index.html · css/style.css
│   ├── dist/app.js                  # bundle (esbuild) — se abre sin servidor
│   ├── src/
│   │   ├── main.js                  # navegación por unidades (hash routing)
│   │   ├── store.js                 # estado compartido de cargas
│   │   ├── physics/                 # constants, electrostatics, dynamics,
│   │   │                            # capacitors, circuits, magnetostatics
│   │   ├── geometry/                # contours (marching squares/tetrahedra), potentialMap
│   │   ├── ui/                      # dom, charts, scene3d, player, chargeTable, format
│   │   └── modules/                 # cargas, potencial, trayectoria, capacitor,
│   │                                # energia, circuitos, ohm, magnetico, lorentz
│   └── tests/                       # fisica.test.js, exportar_casos.mjs
├── python/
│   ├── fisica_em/                   # electrostatica, dinamica, capacitores,
│   │                                # circuitos, magnetostatica, constantes
│   ├── tests/                       # test_fisica.py, test_validacion_web.py
│   ├── scripts/generar_figuras.py · notebooks/validacion.ipynb
```

```
└── docs/                          # manual, informe, capturas, figuras
```

3.3 Decisiones de diseño

Tabla 2. *Principales decisiones de diseño.*

Decisión	Justificación
Aplicación web en lugar de escritorio	Se ejecuta en cualquier computadora con navegador, sin instalar nada; WebGL ofrece 3D acelerado y es fácil de demostrar en clase.
Motor físico separado de la interfaz	Facilita las pruebas automáticas, la reutilización y la validación cruzada con Python.
Bundle con esbuild (IIFE)	Genera un único dist/app.js que se abre con doble clic (sin las restricciones de módulos ES con file://).
Three.js con líneas gruesas (Line2)	Las líneas de campo se leen mejor con grosor constante en pantalla y color por |E| o |B|.
Estado compartido (store.js)	Las mismas cargas alimentan los módulos 1, 2 y 3; los módulos se suscriben a cambios (patrón observador).
Componentes de UI mínimos (dom.js)	Sin framework: menor tamaño, arranque inmediato y control total del ciclo de vida de las escenas WebGL (todas se liberan al cambiar de módulo).
Doble implementación (JS + Python)	Dos implementaciones independientes que coinciden son una evidencia fuerte de corrección.

3.4 Flujo de una interacción

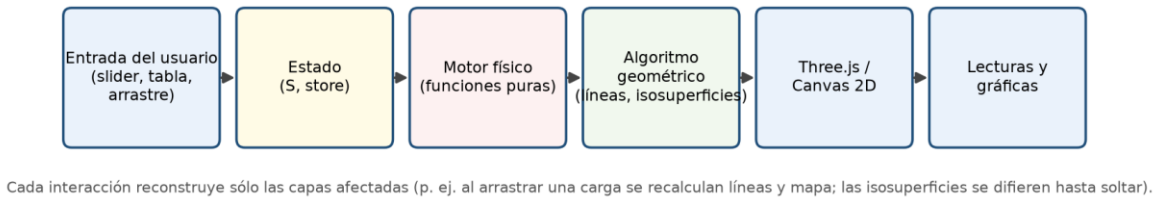


Figura 2. *Flujo de datos de una interacción del usuario hasta la visualización.*

Los controles actualizan un estado local, el motor físico produce datos (campos, trayectorias, corrientes), los algoritmos geométricos los convierten en geometría (polilíneas, mallas) y la capa de presentación los dibuja. Los eventos frecuentes (arrastres, deslizadores) se limitan con *debounce* o con requestAnimationFrame para mantener la fluidez.

3.5 Interfaz y experiencia de usuario

- Navegación por unidades en un menú lateral; cada módulo explica brevemente la física que ilustra.
- Tema oscuro con alto contraste; colores consistentes (rojo = positivo / V > 0, azul = negativo / V < 0).
- Lecturas numéricas con prefijos SI automáticos y verificaciones en verde (balance de potencia, conservación de la energía).
- Escenarios de ejemplo predefinidos en los módulos de partículas y circuitos.
- Diseño adaptable: en pantallas pequeñas el menú se oculta tras un botón y los paneles se apilan.

4. Algoritmos utilizados

4.1 Líneas de campo eléctrico

Cada línea se obtiene integrando dr/ds = ±E/|E| con **Runge–Kutta de cuarto orden** y paso fijo h. Las semillas se colocan sobre una esfera de radio pequeño alrededor de cada carga (espiral de Fibonacci en 3D, circunferencia en 2D) en número proporcional a |q|. Las cargas negativas sólo aportan las líneas que provienen del infinito, en proporción $1 - Q_+/Q_-$ (conservación del flujo de Gauss), para evitar duplicados. La integración termina al llegar a una carga (distancia < 1.5h) o al salir del dominio.

```
para cada carga c con q != 0:
    n = max(4, round(N * |q| / |q|max));  si q < 0: n = round(n * max(0, 1 - Q+/Q-))
    para cada semilla s en esfera_fibonacci(n):
        p = c + r0 * s
        repetir hasta captura o límite:
            p = rk4(p, h, f(p) = signo * E(p)/|E(p)|)
```

4.2 Equipotenciales: marching squares y marching tetrahedra

En 2D se evalúa V en una malla de 200 × 200 puntos y se extraen las isolíneas con **marching squares** (interpolación lineal en las aristas; los casos de silla se resuelven con el valor del centro de la celda). En 3D se evalúa V en una malla de 42^3 puntos y se usa **marching tetrahedra**: cada celda cúbica se divide en 6 tetraedros y en cada uno se generan 0, 1 o 2 triángulos, lo que evita las tablas de casos del cubo completo. Los triángulos se orientan con la normal analítica $\nabla V = -E$ para obtener un sombreado correcto. Los niveles se eligen en escala logarítmica entre dos percentiles de |V|.

4.3 Integración de trayectorias: Dormand–Prince 5(4)

Las trayectorias en campos no uniformes usan el método adaptativo **Dormand–Prince** (RK45) con control de error mixto absoluto/relativo (rtol = 10^{-10}). El paso se ajusta con h ← h·min(5, max(0.2, 0.9·err^(−1/5))). Un límite h_max = T/N garantiza muestras suficientes para la animación. El integrador se valida con la solución analítica y con el teorema trabajo–energía.

4.4 Integrador de Boris

Para el movimiento en campos E y B se ofrece el **integrador de Boris**, estándar en física de plasmas por conservar exactamente |v| en un campo B puro:

$$\vec{v}^{\,-} = \vec{v}_n + \frac{q\vec{E}}{m}\frac{\Delta t}{2}, \quad \vec{t} = \frac{q\vec{B}}{m}\frac{\Delta t}{2}, \quad \vec{s} = \frac{2\vec{t}}{1+t^2} \tag{12}$$

$$\vec{v}' = \vec{v}^{\,-} + \vec{v}^{\,-} \times \vec{t}, \quad \vec{v}^{\,+} = \vec{v}^{\,-} + \vec{v}' \times \vec{s}, \quad \vec{v}_{n+1} = \vec{v}^{\,+} + \frac{q\vec{E}}{m}\frac{\Delta t}{2} \tag{13}$$

La posición se actualiza con la regla del trapecio r_{n+1} = r_n + ½Δt (v_n + v_{n+1}). El esquema es de segundo orden (Figura 5) y es simpléctico en B puro.

4.5 Circuitos

Un circuito de resistencias se representa como un **árbol** cuyos nodos son resistores (R), grupos en serie (S) o grupos en paralelo (P). La resolución es recursiva en dos pasadas: (1) hacia arriba se calcula

la resistencia equivalente de cada subárbol; (2) hacia abajo se reparte la corriente (serie) o el voltaje (paralelo). La complejidad es O(n) en el número de resistores. Para validar el resultado, el árbol se convierte en una lista de aristas y se resuelve con **análisis nodal** (matriz de conductancias G·v = i) en Python; ambos métodos coinciden a precisión de máquina.

```
resolver(nodo, V, I):
    si R:  anotar(V, I)
    si S:  para cada hijo h: resolver(h, I * Req(h), I)
    si P:  para cada hijo h: resolver(h, V, V / Req(h))
```

4.6 Campo magnético: Biot–Savart numérico y líneas de B

Cada espira se discretiza en segmentos (128 para la sonda y 48 para el trazado) con la **regla del punto medio sobre el arco**; el campo es la suma $B = (\mu_0/4\pi)\ \Sigma\ I\ dl \times (r - r_m)/|r - r_m|^3$. El solenoide se modela con hasta 40 espiras (16 para trazar líneas) cuya corriente equivalente I·N/n_modelo representa la hoja de corriente continua. El hilo recto y el dipolo usan expresiones cerradas.

Para las fuentes con simetría axial las **semillas de las líneas se eligen con igual flujo**: toda línea cruza el plano z = 0, por lo que se integra $\psi(\rho) = \int B_z\ \rho\ d\rho$ sobre ese plano y se toman radios con $\psi_k = k\cdot\psi_{max}/(n+1)$. Así la densidad de líneas es proporcional a |B|. Cada línea se traza una vez y se replica en 8 azimuts. El cierre de la línea se detecta al volver al punto de partida.

4.7 Mapas de potencial y color

El mapa de color usa una escala **asinh con signo**, $s = \text{asinh}(V/V_s)/\text{asinh}(V_{hi}/V_s)$, donde V_s y V_{hi} son percentiles de |V|; así se distinguen simultáneamente las zonas cercanas a las cargas y el campo lejano.

4.8 Complejidad y rendimiento

Tabla 3. *Costes y estrategias de rendimiento.*

Tarea	Coste aproximado	Estrategia
Líneas de campo eléctrico	O(L · S · N) (L líneas, S pasos, N cargas)	Paso h = 0.06 m; ≤ 120 líneas
Mapa de potencial 2D	$O(200^2 \cdot N)$	Recalculado en cada arrastre (< 10 ms)
Isosuperficies 3D	$O(42^3 \cdot N + 6 \cdot 42^3)$	Se difiere hasta soltar la carga
Líneas de B de un solenoide	O(L · S · 4 · n_seg)	Malla gruesa para trazar (512 segmentos) y fina para la sonda
Trayectoria RK45 / Boris	O(pasos)	≤ 1200 muestras para animar; Boris con 40 pasos por período

5. Pruebas y validación

5.1 Estrategia

La validación se organiza en cuatro niveles: (1) **pruebas analíticas** en JavaScript (19) y Python (19), que comparan con soluciones cerradas (Coulomb, parábola, Larmor, deriva E×B, hilo infinito, espira, solenoide); (2) **leyes de conservación** (trabajo–energía, balance de potencia, ΣU_i = U total, $\oint B \cdot dl = \mu_o I$); (3) **validación cruzada** (6 pruebas en Python que ejecutan el motor JavaScript y comparan con NumPy/SciPy); y (4) **estudios de convergencia** (Biot–Savart frente a la solución exacta con integrales elípticas; orden del integrador de Boris).

Tabla 4. *Pruebas automáticas del motor JavaScript (npm test: 19/19 superadas).*

Prueba (motor JS)	Criterio
Coulomb: campo y potencial de una carga	Igual a $k\,q/r^2$ y $k\,q/r$ (error rel. $< 10^{-9}$)
Superposición en un dipolo	V = 0 en el plano mediatriz; E central = 2kqd
Energía del sistema de dos cargas	$U = k\,q_1q_2/r$
Líneas de campo del dipolo	Terminan en la carga negativa; 16 de 16 líneas generadas
Trayectoria en E uniforme (RK45)	r, v coinciden con $r_o + v_o t + \frac{1}{2}at^2$ ($< 10^{-8}$)
Teorema trabajo–energía en campo coulombiano	$\Delta K = -q\Delta V$ ($< 10^{-7}$)
Lorentz: ω, deriva E×B con Boris y RK45	Deriva simulada = E/B (< 0.5 %)
Boris en B puro	Conservación de K ($< 10^{-10}$)
Hélice en B uniforme	Radio simulado = $m\,v_\perp/(\lvert q\rvert B)$ (< 0.2 %)
Capacitor y energía	$C = \kappa\varepsilon_o A/d$; U equivalentes ($\frac{1}{2}CV^2$, $\frac{1}{2}QV$, $Q^2/2C$)
Redes de capacitores	ΣU_i = U total en serie y paralelo
Circuito RC	v_C(τ) = 63.2 %; energía disipada = $\frac{1}{2}CV^2$
Circuitos mixtos	Balance de potencia, ley de nodos y de mallas
Biot–Savart: espira y solenoide	Centro y eje frente a fórmulas cerradas (< 0.2 %)
Hilo infinito y ley de Ampère	$\oint B \cdot dl = \mu_o I$ ($< 10^{-9}$)
Espira lejana ≈ dipolo	Diferencia < 2 % a $r \gg R$
Marching squares / tetrahedra	Isolínea e isosuperficie de una carga son circunferencia y esfera (< 0.5 % y 2 %)

5.2 Resultados numéricos de validación

Tabla 5. *Resultados de validación (generados por python/scripts/generar_figuras.py).*

Magnitud verificada	Resultado	Referencia
Parábola de un electrón en E uniforme: error relativo máximo	1.5e-15	Solución analítica
Órbita circular: deriva relativa de la energía en 3 vueltas	2.4e-10	Conservación de la energía
Órbita circular: máxima desviación del radio (m)	1.7e-10	r = 1 m
Radio de Larmor de la hélice (m): simulado	9.040767e-03	teórico 9.040767e-03
Deriva E×B (m/s): simulada	10000.0000	teórica 10000.0000
Orden de convergencia medido del integrador de Boris	1.996	2 (teórico)
Energía en B puro con Boris (32 pasos/período, 20 períodos)	3.3e-15	Conservación exacta
Espira (eje): error relativo máximo de Biot–Savart	6.0e-10	Fórmula cerrada
Biot–Savart fuera del eje con 128 segmentos	1.0e-09	Integrales elípticas (SciPy)
Solenoide (centro): error relativo	2.9e-05	$\mu_o NI/\sqrt{(L^2+4R^2)}$
Circuito mixto: diferencia reducción vs. análisis nodal	6.4e-16	Precisión de máquina
Circuito mixto: balance de potencia ΣP / P_fuente	1.000000	1
RC (10τ): energía disipada / energía almacenada	1.00009	1
RC (10τ): energía de la fuente / CV^2	0.99995	1

5.3 Evidencia gráfica

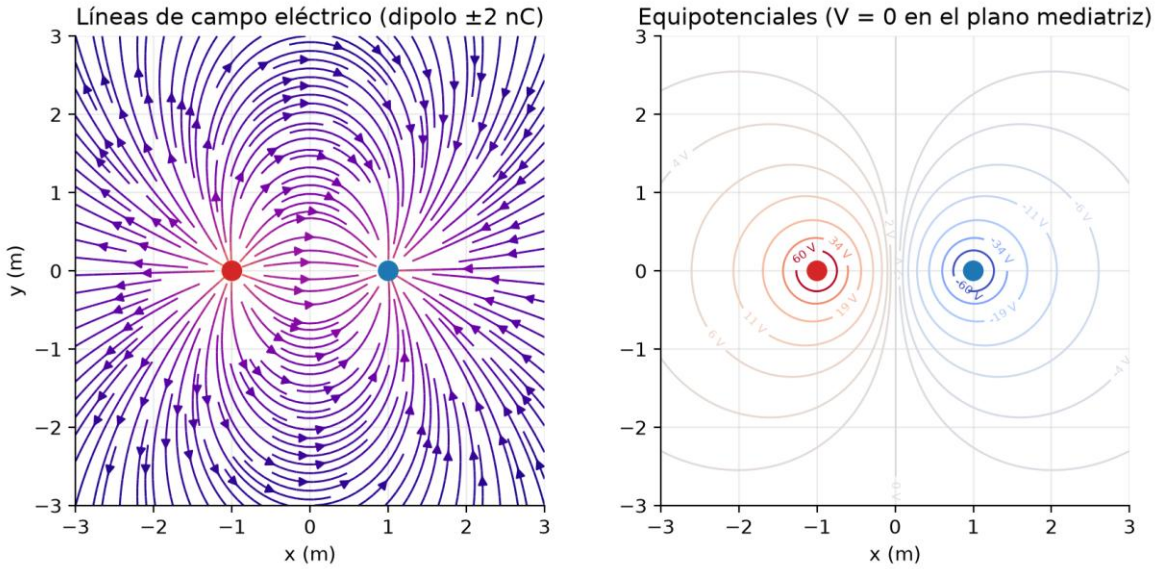


Figura 3. *Dipolo ±2 nC calculado con el paquete Python: líneas de campo y equipotenciales (V = 0 en la mediatriz).*

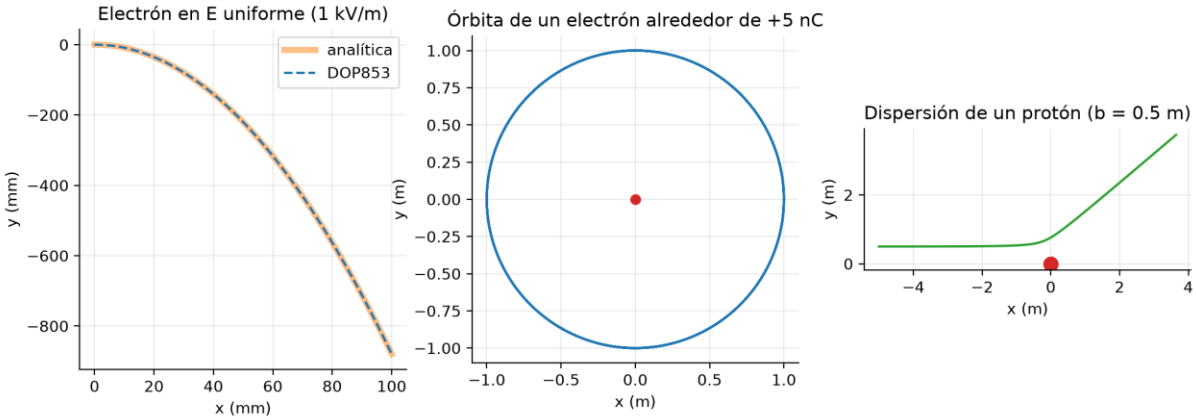


Figura 4. *Trayectorias en campos eléctricos: parábola numérica frente a analítica, órbita circular y dispersión de un protón.*

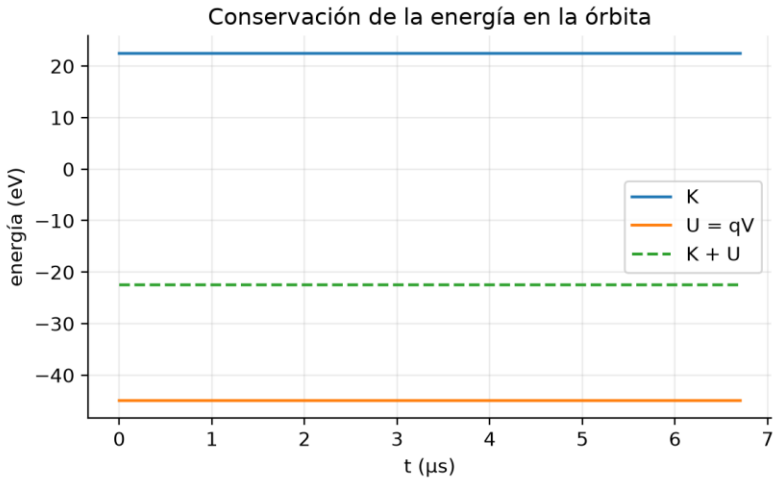


Figura 5. *Conservación de la energía mecánica en la órbita de un electrón (K + U constante).*

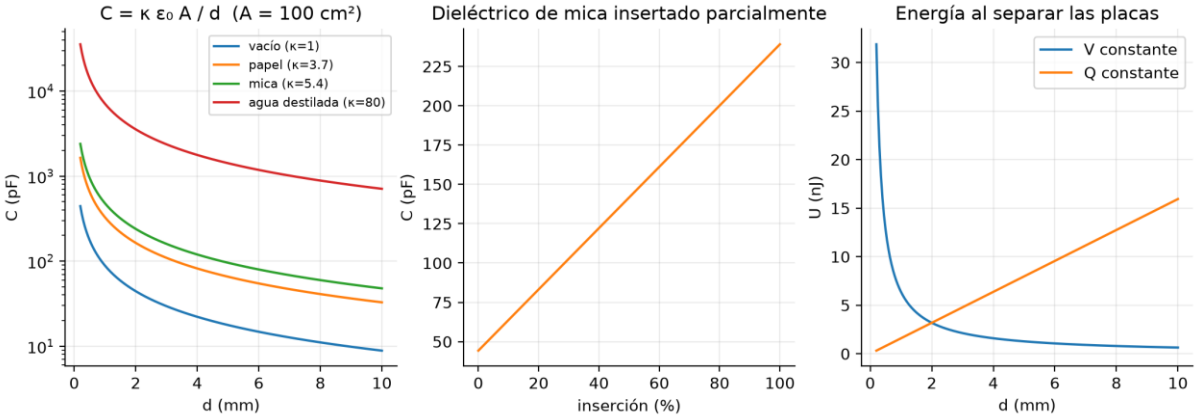


Figura 6. *Capacitancia frente a separación y material, inserción parcial y energía con V o Q constante.*

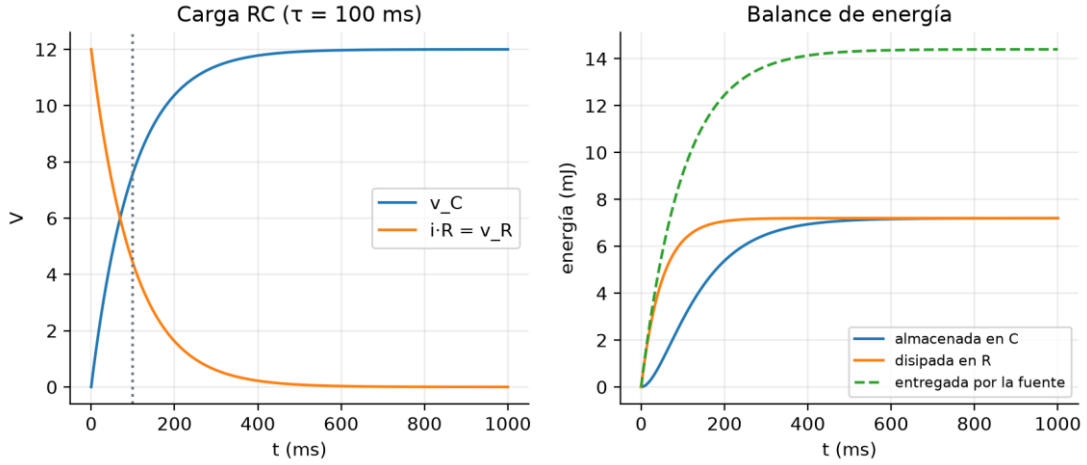


Figura 7. *Carga de un capacitor: tensiones y balance de energía (la mitad de la energía entregada se disipa).*

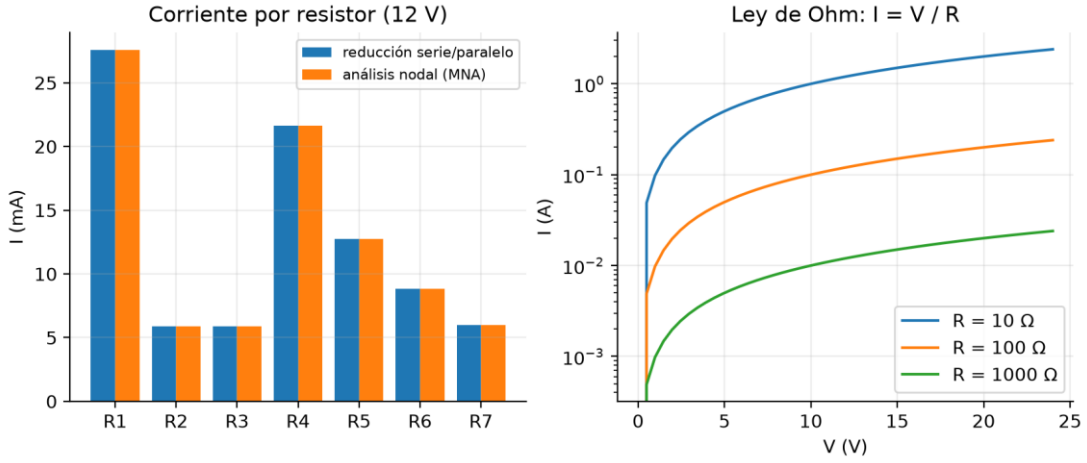


Figura 8. *Corrientes por resistor (reducción serie/paralelo frente a análisis nodal) y característica I–V.*

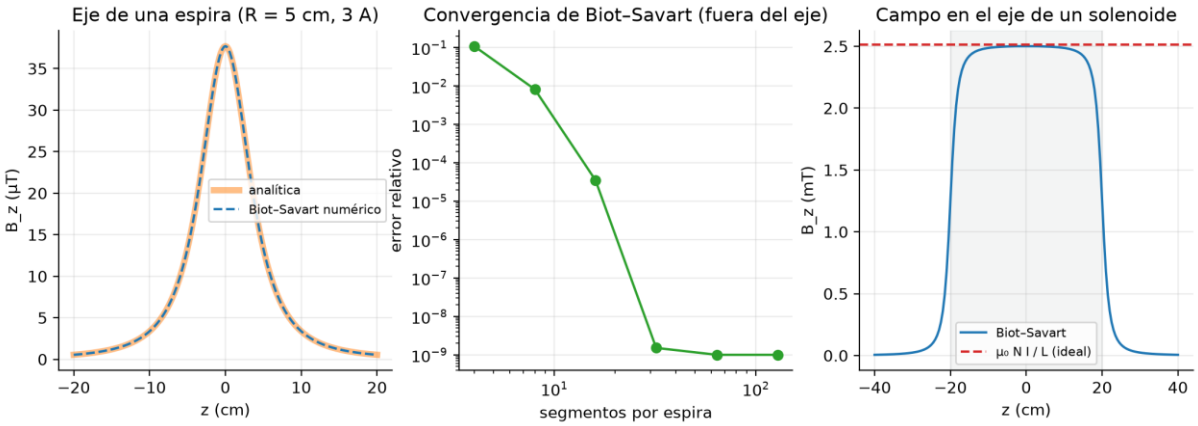


Figura 9. *Campo en el eje de una espira, convergencia de Biot–Savart con el número de segmentos y campo de un solenoide.*

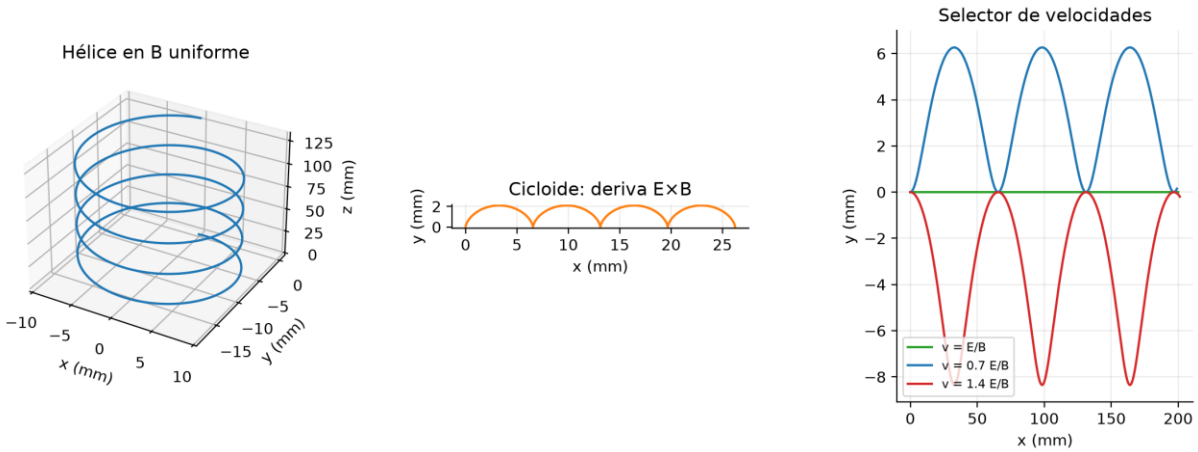


Figura 10. *Hélice, cicloide por deriva E×B y selector de velocidades (v < E/B, v = E/B, v > E/B).*

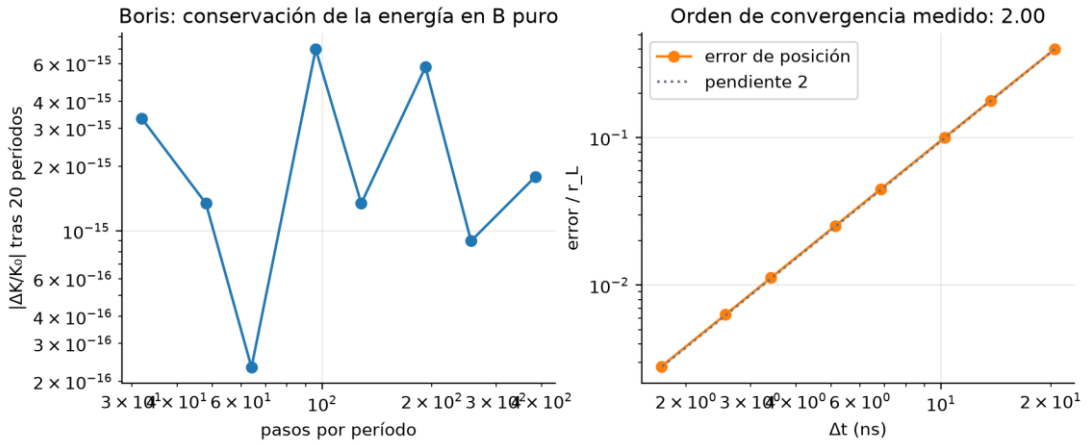


Figura 11. *Integrador de Boris: conservación de la energía y orden de convergencia 2.*

5.4 Validación cruzada JavaScript ⇄ Python

El script `simulador-web/tests/exportar_casos.mjs` ejecuta el motor JavaScript sobre casos fijos (campo de tres cargas, trayectoria de un electrón con E uniforme y campo coulombiano, Lorentz con Boris y RK45, capacitores, un circuito mixto de 7 resistores y campos magnéticos de una espira, un solenoide y un hilo) y los imprime en JSON. Las 6 pruebas de `python/tests/test_validacion_web.py` recalculan cada caso con `fisica_em` y comparan: campos

y potenciales coinciden a 10^{-9}, la trayectoria RK45 a 10^{-6}, las corrientes del circuito a 10^{-9} y el campo magnético a 10^{-9}. Como ambas implementaciones están escritas de forma independiente (distinta tecnología e integradores), la coincidencia es una evidencia sólida de la ausencia de errores de modelado.

```
> cd simulador-web && npm test        # 19 pruebas del motor JS      -> 19 superadas
> cd python && py -m pytest           # 19 analíticas + 6 cruzadas    -> 25 superadas
```

5.5 Verificación de la interfaz

Cada módulo se inspeccionó en un navegador real (Chromium con WebGL): carga sin errores en consola, arrastre de cargas con el ratón, cambio de vistas 2D/3D, reproducción de trayectorias, y lecturas numéricas coherentes con las verificaciones anteriores. Las capturas del manual se generaron con la propia aplicación.

6. Resultados: el simulador en funcionamiento

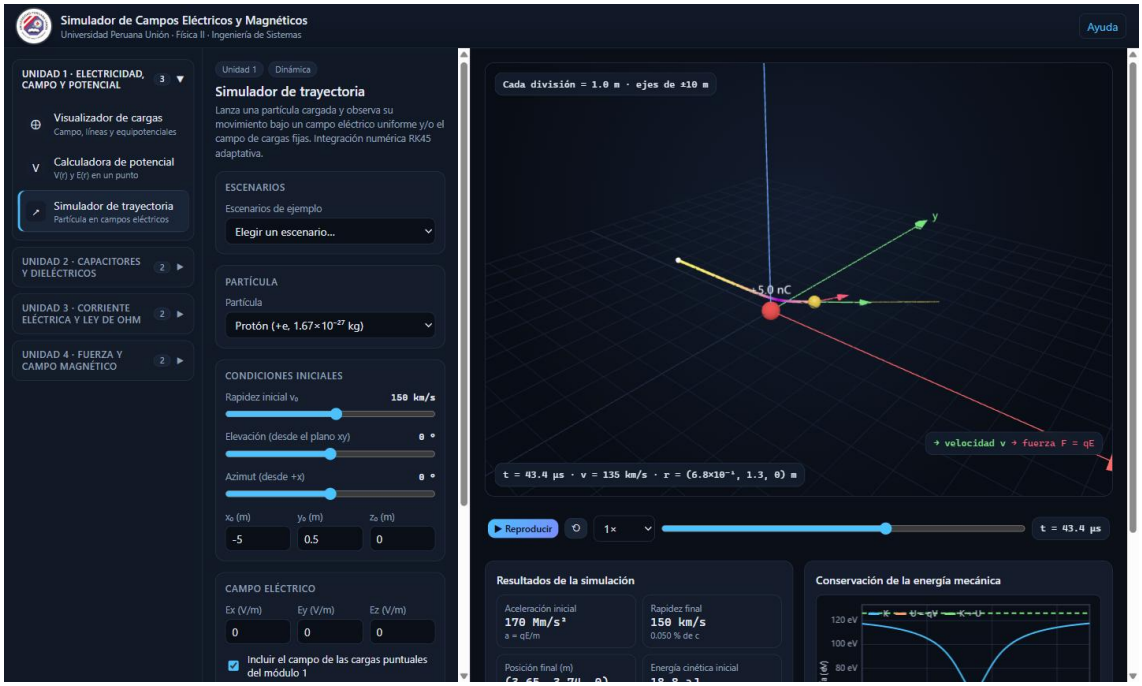


Figura 12. *Dispersión de un protón por una carga fija: la energía mecánica se conserva (error ΔK + ΔU del orden de 10^{-8}).*

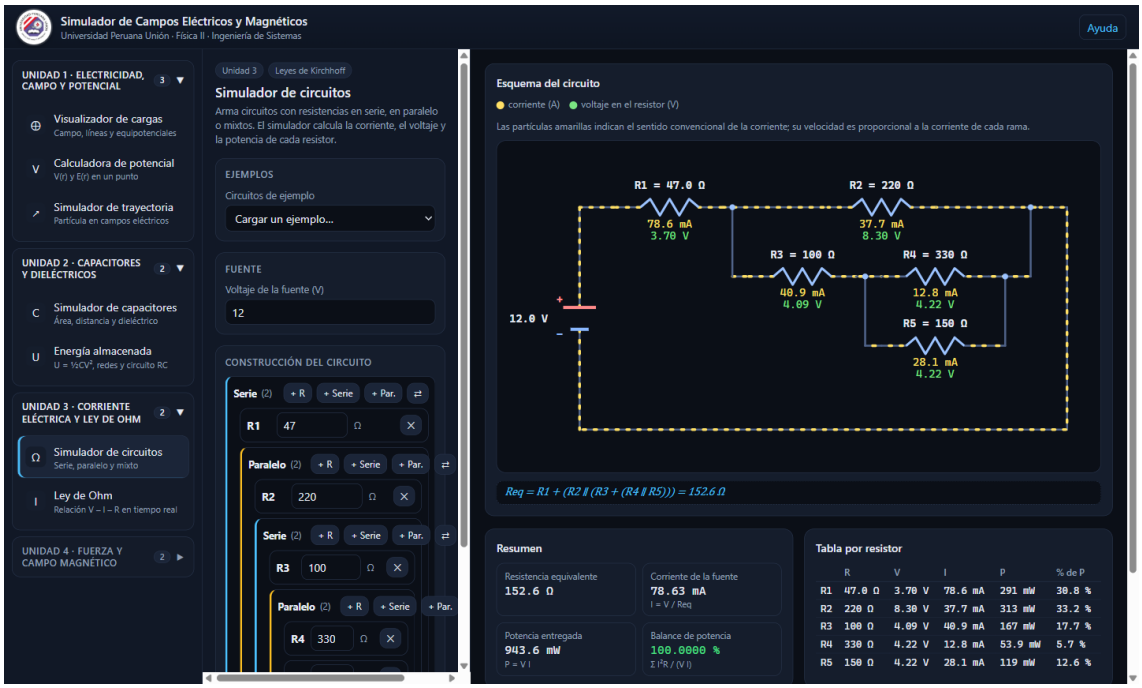


Figura 13. *Circuito mixto anidado con esquema animado, tabla por resistor y balance de potencia 100 %.*

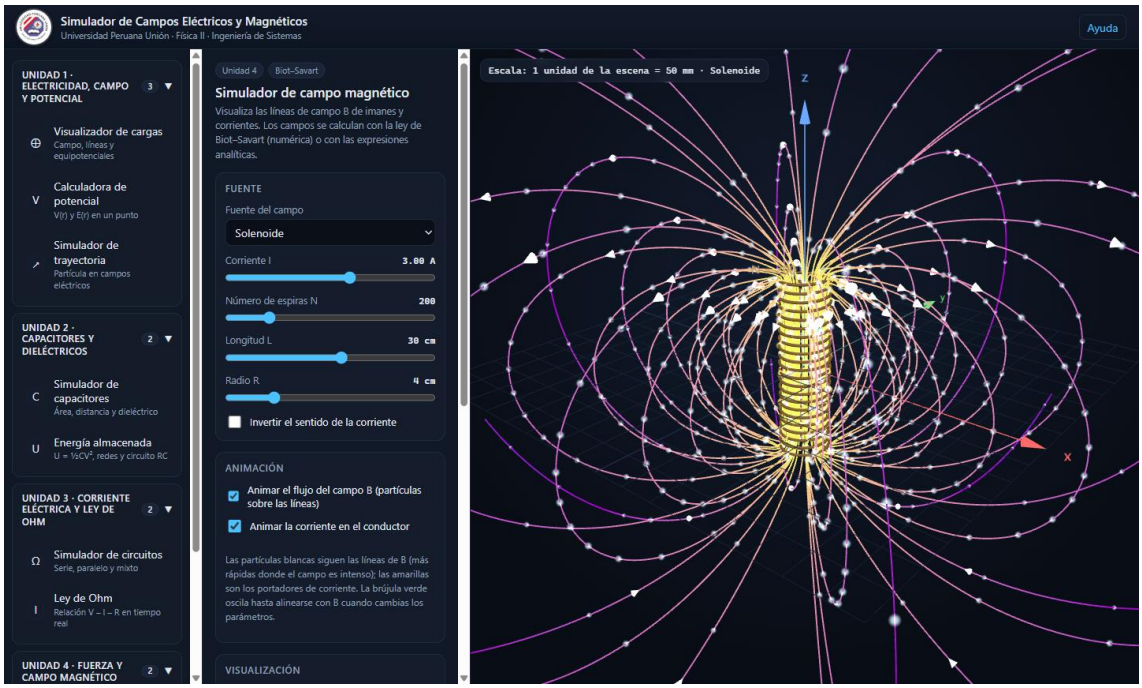


Figura 14. *Campo magnético de un solenoide de 200 espiras (líneas cerradas coloreadas por |B|).*

7. Limitaciones y trabajo futuro

- El simulador es **no relativista**: se advierte al usuario si v > 0.1c.
- Las partículas no se afectan entre sí (campo de las cargas fijo); no se modela radiación.
- Los circuitos se limitan a resistores en serie/paralelo (árboles). Las mallas arbitrarias (p. ej. puente de Wheatstone) pueden añadirse con el análisis nodal ya implementado en Python.
- El capacitor es de placas paralelas ideales (sin efectos de borde); se podrían añadir geometrías cilíndrica y esférica.
- Los campos E y B del módulo de Lorentz son uniformes; se podrían añadir una botella magnética y un ciclotrón con campo alterno.
- El solenoide se discretiza con un máximo de 40 espiras (hoja de corriente equivalente), por lo que el campo muy cerca del bobinado es aproximado.
- Mejoras de ingeniería: web workers para cálculos pesados, exportación de datos en CSV y de escenas como imágenes.

8. Conclusiones

1. Se implementaron los nueve módulos solicitados para las cuatro unidades, con visualización 2D/3D en tiempo real e interacción directa (arrastre de cargas, escenarios de ejemplo, reproducción de trayectorias).
2. La separación entre el motor físico y la interfaz permitió probar los modelos de forma automática: 19 pruebas en JavaScript y 25 en Python, todas superadas.
3. La validación cruzada entre dos implementaciones independientes y las soluciones analíticas respalda la corrección: errores del orden de 10^{-15} en la parábola y de 10^{-7} o menores en radio de Larmor, deriva E×B y campo magnético.
4. El integrador de Boris conservó la energía en campo magnético y mostró el orden de convergencia teórico (1.996 ≈ 2), mientras que RK45 aporta alta precisión en campos arbitrarios.
5. El proyecto integra competencias de Ingeniería de Sistemas: modelamiento matemático, programación científica, arquitectura modular, UX/UI, visualización de datos, simulación y trabajo colaborativo con control de versiones.

Competencias desarrolladas

Tabla 6. *Competencias de Ingeniería de Sistemas.*

Competencia	Evidencia en el proyecto
Modelamiento matemático	Algoritmos para resolver ecuaciones físicas: RK4, Dormand–Prince, Boris, Biot–Savart, reducción y análisis nodal.
Programación científica	Motor numérico en JavaScript y NumPy/SciPy en Python; pruebas contra soluciones analíticas.
Arquitectura de software	Capas físico/geométrico/presentación, estado compartido con patrón observador, bundle reproducible.
UX/UI	Interfaz en español, lecturas con unidades, verificaciones visibles, ejemplos guiados y diseño adaptable.
Visualización de datos	Líneas de campo, isosuperficies, mapas de potencial y gráficas interactivas 2D/3D.
Simulación	Animaciones en tiempo real de partículas, corrientes y capacitores.
Trabajo en equipo	Estructura de repositorio y README listos para colaboración en Git/GitHub.

Referencias

- Young, H. D. y Freedman, R. A. *Física Universitaria con Física Moderna, Vol. 2* (Sears–Zemansky). Pearson.
- Serway, R. A. y Jewett, J. W. *Física para Ciencias e Ingeniería, Vol. 2*. Cengage.
- Griffiths, D. J. *Introduction to Electrodynamics*. Cambridge University Press.
- Dormand, J. R. y Prince, P. J. (1980). A family of embedded Runge–Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1), 19–26.
- Boris, J. P. (1970). Relativistic plasma simulation — optimization of a hybrid code. *Proc. Fourth Conf. on Numerical Simulation of Plasmas*.
- Birdsall, C. K. y Langdon, A. B. *Plasma Physics via Computer Simulation*. CRC Press.
- Lorensen, W. E. y Cline, H. E. (1987). Marching cubes. *SIGGRAPH Computer Graphics*, 21(4).
- Ho, C.-W., Ruehli, A. y Brennan, P. (1975). The modified nodal approach to network analysis. *IEEE Trans. Circuits and Systems*, 22(6).
- Documentación de Three.js (https://threejs.org), NumPy (https://numpy.org) y SciPy (https://scipy.org).